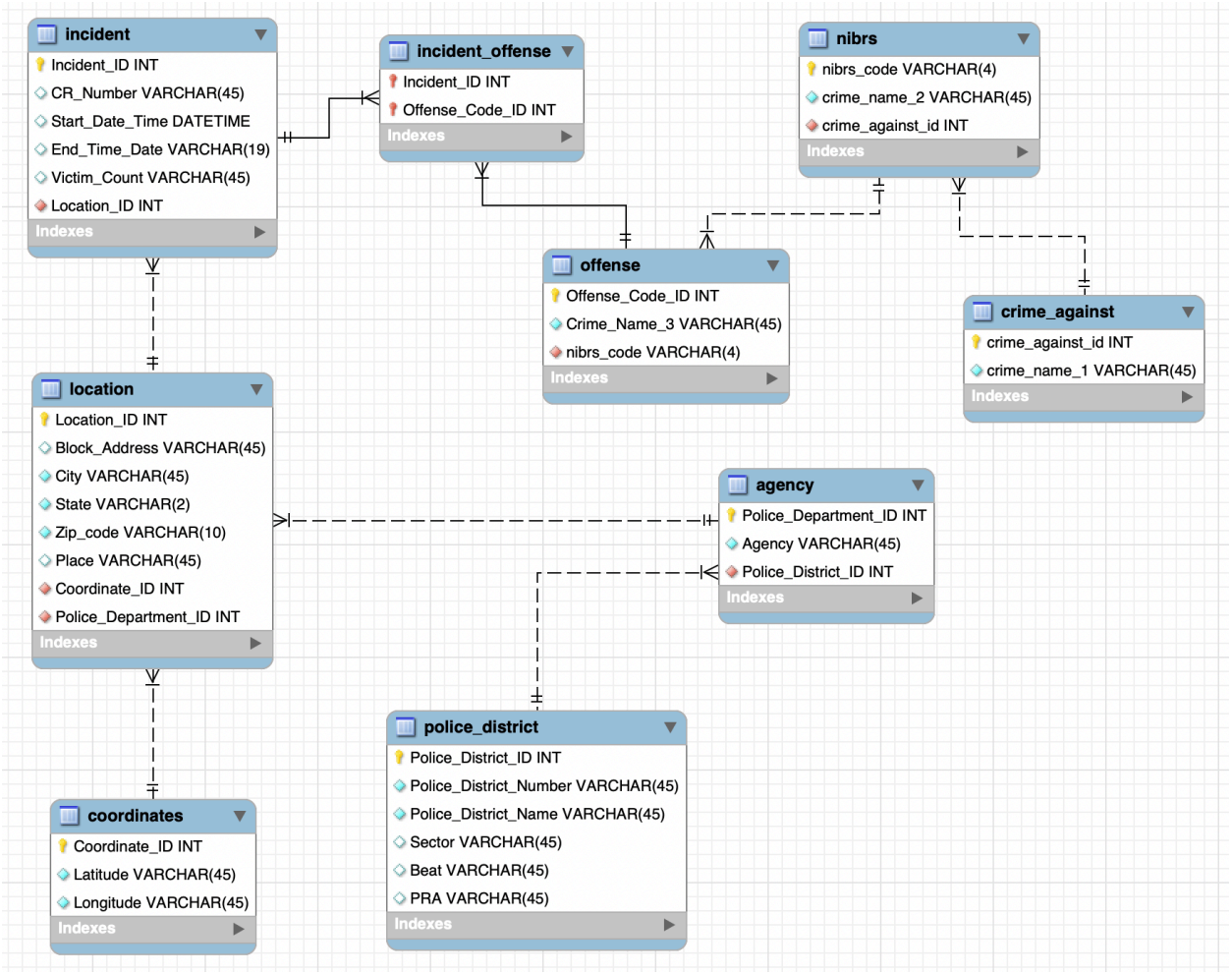
INST 327 - Database Design and Modeling
Section 0302
Team 7 Final Report
Montgomery County Crime Database
Members: Aaron Atayan, Ethan Nalima, Shiv Kasondra, Anthony Zhou, Shazab Chaudhry

1. Introduction

- Describe the Montgomery County Crime domain
- Explain the motivation for the database
- Introduce how the database fulfills information needs
- Mention target audience (homebuyers, police, advocates)

The Montgomery County Crime dataset is a large and frequently updated database that houses public data regarding reported crime and statistics in the county. The dataset includes information on the types of crime committed, the perpetrator of the crime, the location of the crime, and the specific agency investigating the crime. The dataset contains records of crimes reported after July 1st, 2016, until the current date, and its classification is derived from the National Incident-Based Reporting System (NIBRS) of Criminal Justice Information Services (CJIS) Division Uniform Crime Reporting (UCR) Program. Our goal was to narrow down this large dataset and create a more comprehensive and coherent relational database that makes it easier to analyze crime patterns over time and across jurisdictions. It takes into account common questions such as how crime varies by city or police district. We are targeting homeowners, police, and county officials, and also community advocates who can benefit from clear and up-to-date information about crime trends in Montgomery County.

2. Database Description (2-3 pages)



Incident Table

- Incident_ID (PK)
- CR_Number
- Dispatch_Date_Time
- Start_Date_Time
- End_Time_Date
- Victim_Count
- Location_ID (FK)

Each incident links to its location, and the offenses related to that incident are captured with the linking table

Incident_has_Offense (Linking Table)

The purpose of this table is to solve the many-to-many relationship between incidents and offenses. One incident can involve multiple offenses, and one offense code can appear in many incidents.

Offense Table

This contains the definitions of crime offenses, which are:

- Offense_Code_ID (PK)
- Offense names like Crime_Name_3, etc.
- Nibrs_code (FK)

nibrs Table

This maps the national reporting standard values, which are:

- nibrs_code (PK)
- Offense names like Crime_Name_2, etc.
- Crime_against_id (FK)

crime_against Table

This table defines high-level categories like “Crimes Against Persons,” Crimes Against Property,” etc.

Location Table

This table represents the physical location of an incident. This table has:

- Location_ID (PK)
- Block address, street components
- City, State, Zip
- Place
- Police_Department_ID (FK)

We maintained this table as a single, unified table rather than splitting out coordinates, as it simplified writing the query and avoided redundant joins.

Agency

This table lists the agency responsible for handling the incident. This table has:

- Police_Department_ID (PK)
- Agency Name
- Police_District_ID (FK)

Police_District

This table stores the structure of Montgomery County police districts:

- Police_District_ID
- District number and name
- Sector, Beat, PRA information

Normalization Decisions

Our final ERD is a 3NF. It has no repeating groups, no transitive dependencies, no compound fields, a linking table only used for many-to-many relationships, and has lookup tables. This design makes sure that we have accurate reporting, faster queries, and the potential for easier expansion in future updates.

Why We Removed The Coordinates Table

Originally, we had a coordinates table that stored coordinates. After reviewing the dataset and the proposal feedback, we realized that storing coordinates was unnecessary and created unnecessary joins. The coordinates table did not add meaningful normalization benefits; each coordinate pair maps directly to one location. Removing this table reduced schema complexity and improved the query readability.

Why We Changed Place to Place_Type

The original Place table mixed physical locations with types of locations. To avoid confusion, we redesigned the structure where Location is the real-world address where an incident occurred, and the Place_type is the categorical description, like residence, store, or parking lot. This change makes the query cleaner and avoids storing redundant labels across the records.

Primary & Foreign Keys (NOT DONE)

Primary keys identify each record in its own unique way. Like Indcident_ID, and Offense_Code_ID.

Foreign keys enforce referential integrity. These constraints ensure that every incident, offense, location, and district is valid and properly linked.

Indexes Added for Performance

To support filtering and joining, we added indexes on:

- Foreign key columns like Location_ID, Police_District_ID, Offense_Code_ID)
- Offense code in the offense table
- District number in Police_District
- CR_Number for rapid lookup of incident entries

These indexes greatly improve query speed for large datasets.

Temporal Columns (Year, Month, Day_of_Week, Hour)

We added temporal attributes to our Incident table to support common analysis tasks like Crime frequency by year, peak crime hours, and weekday vs weekend crime patterns.

This improves performance by eliminating the need for repeated data extraction in queries.

Data Filtering (2020-2023, Top 5 Cities)

We limited our dataset to the year 2016 (July month), which ensured manageable and most recent crime data from the records. We also included the top 5 cities in Montgomery County. This produced a cleaner dataset and allowed us to design a realistic database for a semester-long project.

Data Quality Measures

We ensured data quality by removing incidents with missing addresses or dates, standardizing city names, validating district numbers, and ensuring foreign key values exist before insertion.

Data Cleaning

Cleaning included converting date strings to proper DATETIME format, removing records without usable location values, standardizing offense names and categories, and ensuring coordinates exist for all locations. Also within the dating cleaning , particularly with the nibrs table, when implementing the data into the database , we had values that according to sql were duplicated so we created added a 1 and a 2 to help separate the values to ensure the import would go though successfully

Query Name	Requirement A	Requirement B	Requirement C	Requirement D	Requirement E
geographic_crime_patterns	X	X	X		X
incident_offense_details	X			X	
high_victim_incidents	X	X			
incidents_per_district	X		X		
incidents_with_complete_time	X	X			
Total	5	3	2	1	1

Changes from Original Design

Our database went through several major changes from what we originally made and were proposing to do. The biggest change was the way we handled location data. In our initial drafts, we had separate tables for Coordinates and Place, but when we actually started building our database, we realized this was creating a lot of duplicate information. We ended up combining everything into one LOCATION table that includes block addresses, cities, zip codes, latitude, and longitude. This made more sense and kept us from storing the same information in multiple places.

The Incident table also had a lot of changes. Originally, we had planned three separate columns: Crime_Name1, Crime_Name2, and Crime_Name3 in order to store different crime types for each incident. We, however, reassessed this and did not go with the idea because it wouldn't follow the normalization rules. We instead created an Incident_Offense junction table in order to properly handle the many-to-many relationships between the incidents and offense types. We also simplified our dates and time fields. Instead of having Dispatch_DateTime, Start_Date_Time, and End_Date_Time all as separate datetime fields, we added the Year, Month, Day_of_Week, and Hour fields for the purpose that it would be easier to filter by time in our queries.

Another thing was that we had to be more realistic about the scope of our data, before we would include all incidents from 2020 to 2023, but once we saw how much data that actually was, we had to narrow it down. We focused on a sample that was still big enough to be meaningful but manageable for our team to work with. We also added a place_type table that we didn't have in the original proposal. This let us categorize locations as residential, commercial, or other types, which was really useful for analyzing where different crimes happen.

Lastly, another big change we made was to the crime categories. The Crime_Type table we initially had was too simple for the BIBRS system. The NIBRS coding system had a hierarchy, so we decided to create separate tables for Offense, NIBRS, and crime_against to properly represent the crime classifications. This made much more sense in terms of how law enforcement actually categorizes crime.

Database Ethics Considerations

Since the database uses publicly available Montgomery County crime data, we made sure that the database included incidents from all areas of the county, so there are no neighborhoods that are ignored or overly emphasized.

To protect privacy, our design includes only incident-level information that is from the open data portal and does not include names, exact addresses, or any other sensitive personal details. By limiting the attributes to only what is necessary for analysis, our database reduces the risk that specific individuals can be identified.

Another big part of the considerations was bias in crime data. Reporting and enforcement may be higher in certain areas, which can make those communities appear more dangerous even

when the underlying behavior is similar. **The results are presented with these limitations in mind, so users understand that the counts reflect policing and reporting patterns, not inherent qualities of Montgomery County residents.**

It is also fair use as the dataset is explicitly published for public access and use through the county's open data portal. The system only uses the data for aggregate analysis and visualization.

Lessons Learned

The most significant thing we learned was that a good database design requires a lot more planning and teamwork than we initially expected. When we first started, we thought we could simply jump in and start creating the tables, but we quickly encountered problems. Our early attempts at storing crime information had a lot of redundancy, and we kept finding inconsistencies and other issues pertaining to our database. With the help of our TA, we were able to go back and properly normalize everything the correct way. This would be very important, as if we didn't do this, we'd run into even more problems later.

Within the MySQL Workbench, we also had some trouble. With the foreign key constraints, the tool would sometimes not recognize our relationships correctly, or we'd get errors that we didn't understand before. And when we tried to first reverse-engineer our ERD from the actual database, things wouldn't always match up. We learned to keep our SQL scripts and ERD diagrams in sync so that we'd always have to go in and double-check that everything in the schema matched what we wanted/expected.

Cleaning the crime data was also not as straightforward as we might have expected. The raw data from the Montgomery County dataset had an array of inconsistencies that we had to deal with. For example, location names were spelled differently, there were missing values everywhere, and we were not familiar with the formatting. We had to write scripts to standardize things before we could even load the data into our tables. For example, we created lookup tables to make sure city names were always spelled the same way. The whole process taught us that working with real data is messy and that you spend a lot of time just getting the data ready before you can do anything interesting or even draft any conclusions or analysis on it.

Lastly, we learned how much our design could change based on the queries we needed to write. We thought once we had our tables set up, we'd be done with the design work, but in reality,

when we started writing queries to answer questions, we kept finding ways our schema could be improved upon or changed. For example, filtering by month or day of the week was really slow until we added those as separate fields instead of calculating them from the datatime every time. In conclusion, we all agreed that it’s better to fix things as you go than to stick with a design that makes everything harder.

Potential Future Work

For potential future work, the database could be expanded in several meaningful ways. Firstly, the dataset could be extended to include all available years back to 2016, which can be used for longer-term trend analysis. Additional contextual datasets, such as the area's weather and demographic data, could be integrated to study how the environment and population characteristics correlate with crime patterns. Another potential use case for this could be using an application such as QGIS or a web interface to allow people to run queries on an actual map and view different visualizations. Lastly, it can be a useful asset for predicting new trends that may occur and also improving the data collection process itself.

Work Cited

Crime | Open Data Portal. (n.d.). Data.montgomerycountymd.gov. https://data.montgomerycountymd.gov/Public-Safety/Crime/icn6-v9z3

D'Paul Nibber. (2021, November 3). *Montgomery Council Approves Police Transparency Measure*. Conduit Street. https://conduitstreet.mdcounties.org/2021/11/03/montgomery-council-approves-police-transparency-measure/

Montgomery County Data | Open Data Portal. (n.d.). Tyler Data & Insights. https://data.montgomerycountymd.gov/